

PhantomOS 终极架构

行业首个 EL2 Hypervisor + 双芯片 iOS 兼容层 目标：做到行业没人做到的事——从硬件虚拟化层面实现完全不可检测

一、核心理念

一句话：游戏和反作弊跑在虚拟机里面，我们的代码跑在虚拟机外面。ARM 硬件架构保证里面看不到外面。

这不是 hook，不是伪装，不是藏。是物理上不可能被检测。

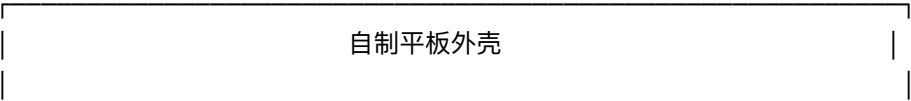
二、为什么行业没人做到

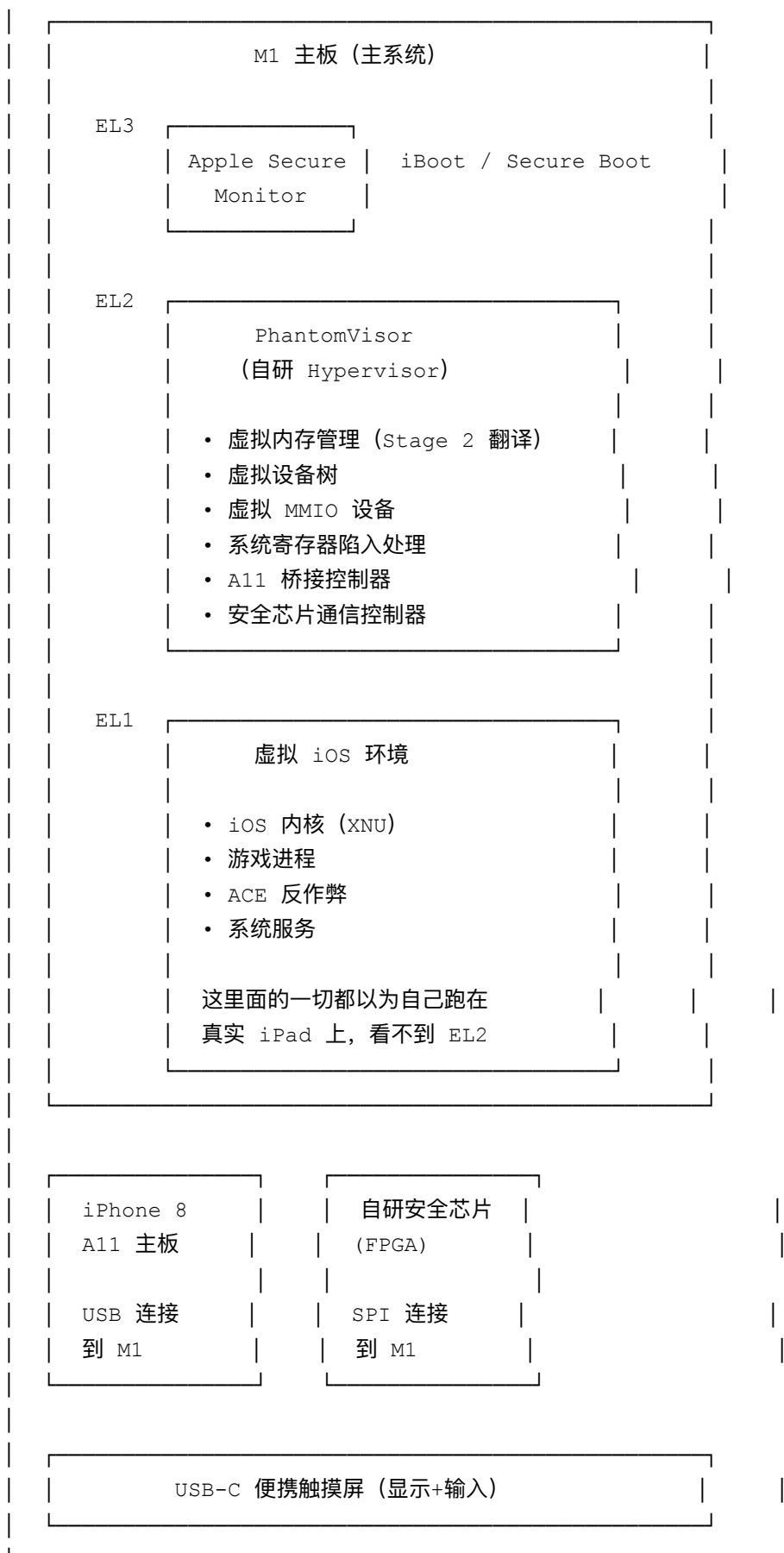
现有方案和它们的天花板：

方案	代表产品	天花板
DYLD_INTERPOSE hook	PlayCover	扫 image list 就暴露
ObjC Runtime Swizzle	各种越狱插件	检查 IMP 地址就暴露
进程外内存读写	传统外挂	被扫描进程外行为就暴露
iOS 虚拟机	vphone-cli / Corellium	FairPlay 过不了，App Store 用不了
macOS 兼容层	PhantomOS 当前架构	87 项检测永远追不完

没有人做过的事：在 Apple Silicon 上跑自研 EL2 Hypervisor，构造一个硬件级虚拟 iOS 环境，配合真实 iPhone 硬件完成所有认证。

三、硬件架构





三块芯片，三个角色

芯片	角色	一句话
M1	计算 + Hypervisor 宿主	提供算力，跑 PhantomVisor
A11	苹果认证代理	提供真实 SEP 签名，解决 FairPlay / App Attest
FPGA	项目保护	许可验证 + 规则加密存储，没芯片系统不启动

四、ARM 特权级架构——为什么不可检测

EL3 – Secure Monitor	
Apple 的安全启动链，我们不碰	
EL2 – Hypervisor	← 我们在这里
直接控制硬件虚拟化扩展	
管理 Stage 2 地址翻译	
拦截 EL1 的所有敏感操作	
EL1 无法感知 EL2 的存在	
EL1 – OS Kernel	← 游戏+ACE在这里
iOS 内核 / 游戏 / 反作弊	
以为自己跑在裸机上	
访问硬件会被 EL2 陷入拦截	
无法执行任何指令来检测 EL2	
EL0 – User Space	
游戏 App / ACE SDK	
完全被 EL1+EL2 控制	

ACE 在 EL0/EL1 运行。PhantomVisor 在 EL2 运行。ARM 架构规定低特权级无法访问高特权级的状态。这是硬件保证，不是软件技巧。

ACE 能做的极限检测：

- 读系统寄存器查虚拟化？→ EL2 陷入，PhantomVisor 返回”没有虚拟化”
- 查内存看有没有 Hypervisor？→ Stage 2 翻译，PhantomVisor 的内存 EL1 映射不到
- 测时间差检测虚拟化？→ PhantomVisor 控制计时器寄存器，抹平延迟

- 查 MMIO 设备? → PhantomVisor 构造虚拟设备, 返回真实 iPad 的值

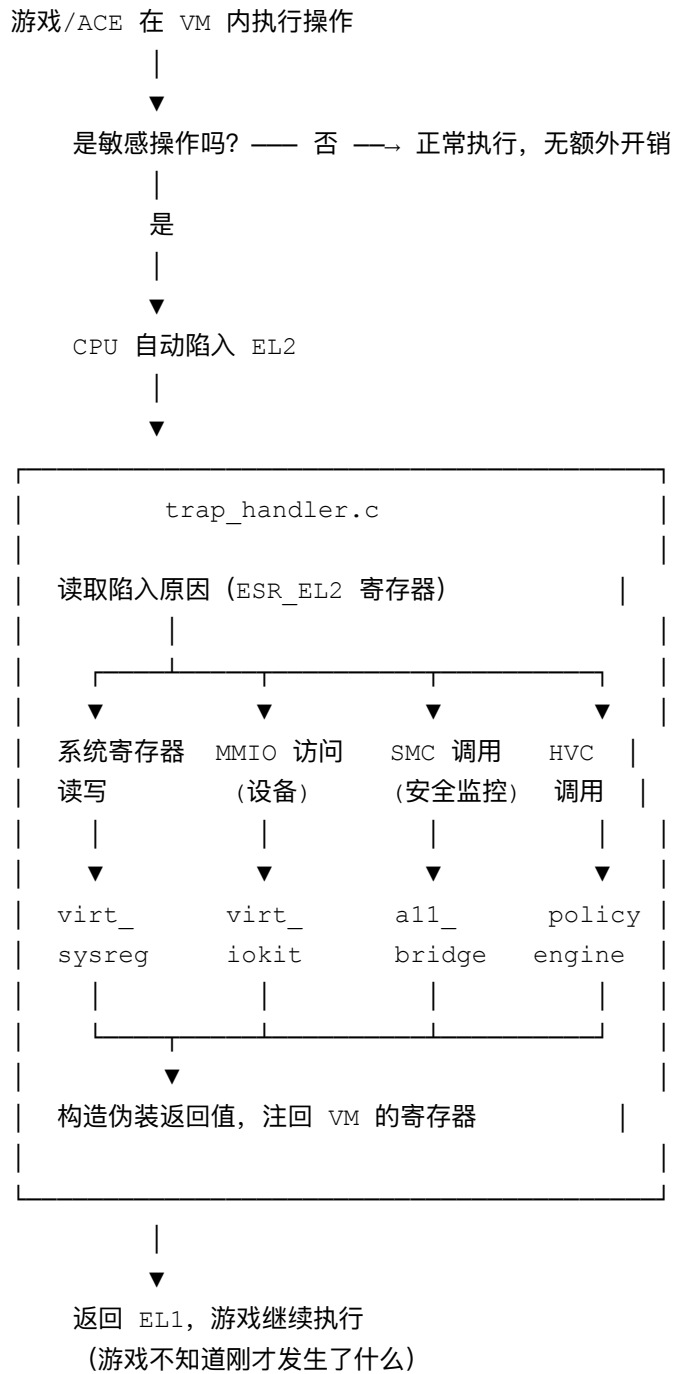
五、PhantomVisor 软件架构

5.1 模块总览

```
PhantomVisor (EL2)
|
├── core/
|   ├── vmm.c                // 虚拟机管理器主循环
|   ├── stage2_mmu.c         // Stage 2 地址翻译 (控制 VM 能看到什么内存)
|   ├── trap_handler.c       // EL1→EL2 陷入处理 (所有敏感操作的统一入口)
|   └── timer.c              // 虚拟计时器 (抹平虚拟化时间差)
|
├── devices/
|   ├── virt_sysreg.c        // 虚拟系统寄存器 (MIDR_EL1, REVIDR_EL1 等)
|   ├── virt_iokit.c         // 虚拟 IOKit 设备树 (返回 iPad 设备信息)
|   ├── virt_gpu.c           // 虚拟 GPU 参数 (Metal Family, 纹理尺寸等)
|   ├── virt_sensor.c        // 虚拟传感器 (加速度计/陀螺仪模拟数据)
|   ├── virt_net.c           // 虚拟网络栈参数 (TLS 指纹, TCP 特征)
|   └── virt_display.c        // 虚拟显示参数 (分辨率, 刷新率)
|
├── bridge/
|   ├── all_bridge.c          // A11 USB 通信控制器
|   ├── all_fairplay.c        // FairPlay 请求转发到 A11
|   ├── all_attest.c          // App Attest 请求转发到 A11
|   └── all_sep.c             // SEP 签名请求转发到 A11
|
├── security/
|   ├── fpga_comm.c           // FPGA 安全芯片通信
|   ├── license.c             // 许可验证
|   ├── rule_loader.c         // 加密规则加载
|   └── heartbeat.c           // 运行时心跳校验
|
└── policy/
    ├── policy_engine.c       // 策略规则引擎
    ├── syscall_policy.json    // BSD 系统调用伪装规则 (加密存储在 FPGA)
    └── mach_policy.json       // Mach IPC 伪装规则 (加密存储在 FPGA)
```

5.2 陷入处理——PhantomVisor 的核心

当 VM 内的游戏/ACE 执行敏感操作时, CPU 自动陷入 EL2, PhantomVisor 接管:



5.3 关键系统寄存器伪装

ACE 可以通过读 ARM 系统寄存器来识别设备。PhantomVisor 在 EL2 拦截这些读操作:

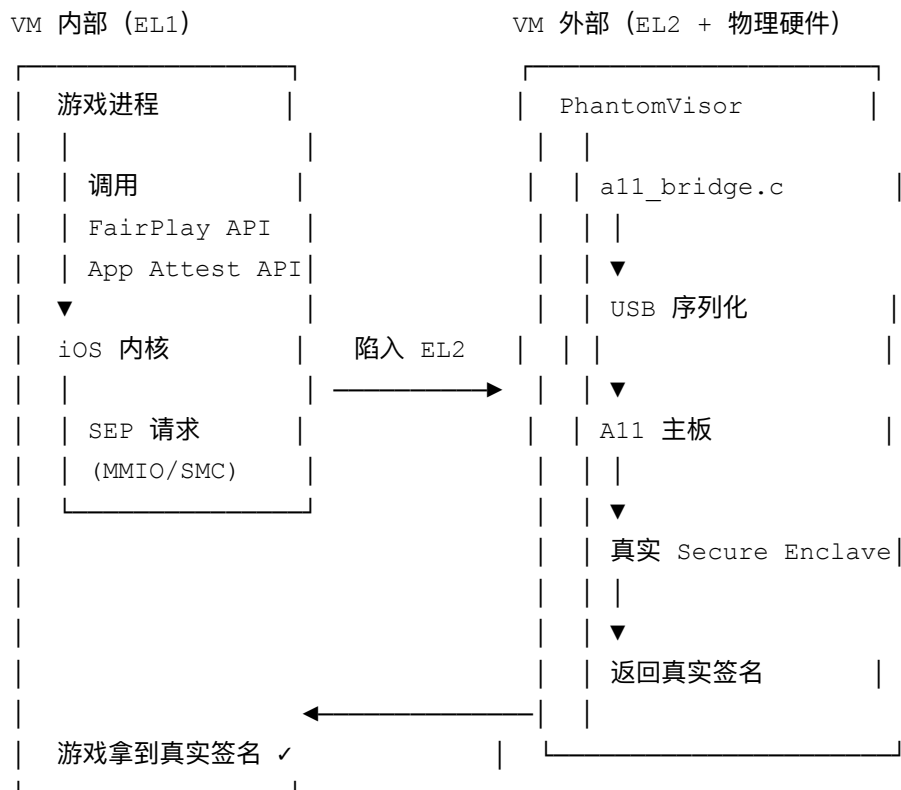
寄存器	真实值 (M1 Mac)	PhantomVisor 返回值 (iPad)	作用
MIDR_EL1	M1 的 CPU ID	A14/M1 iPad 的 CPU ID	CPU 型号识别

REVIDR_EL1	M1 修订版本	iPad 对应版本	CPU 修订版本
CLIDR_EL1	M1 缓存层级	iPad 缓存层级	缓存架构
CTR_EL0	M1 缓存行大小	iPad 缓存行大小	缓存参数
DCZID_EL0	M1 DC ZVA 大小	iPad DC ZVA 大小	数据缓存参数
ID_AA64MMFR*_EL1	M1 内存模型	iPad 内存模型	内存特性

这些寄存器在 EL1 读取时自动陷入 EL2。PhantomVisor 返回 iPad 的值。ACE 拿到的就是 iPad 的 CPU 信息，不是 Mac 的。不需要 hook 任何函数，硬件自动完成。

六、A11 桥接——解决 FairPlay / App Attest

6.1 架构



6.2 FairPlay 解密流程

1. VM 内 iOS 从 App Store 下载加密 App

2. iOS 内核发起 FairPlay 解密请求
3. 请求涉及 SEP 硬件操作 → MMIO 访问陷入 EL2
4. PhantomVisor 捕获请求
5. 通过 USB 转发到 A11 主板
6. A11 的真实 Secure Enclave 执行解密
7. 解密密钥通过 USB 返回
8. PhantomVisor 注入 VM 的 SEP 响应寄存器
9. iOS 内核拿到密钥，解密成功
10. 游戏正常启动

游戏和 ACE 全程在 VM 内部，看到的是：

"我请求了 SEP 解密，SEP 给了我密钥，一切正常"

它不知道这个 SEP 其实是一块 USB 外接的 iPhone 8 主板

6.3 App Attest 流程

1. 游戏调用 `DCAppAttestService.generateKey()`
2. 最终走到 SEP 的密钥生成 → MMIO 陷入 EL2
3. PhantomVisor → USB → A11 SEP 生成真实密钥对
4. 公钥回传，注入 VM
5. 游戏调用 `attestKey()` → 签名请求 → 再次陷入
6. PhantomVisor → USB → A11 SEP 用真实私钥签名
7. 签名结果注入 VM
8. 游戏拿到 Apple 信任链上的真实 attestation → 发给服务器 ✓

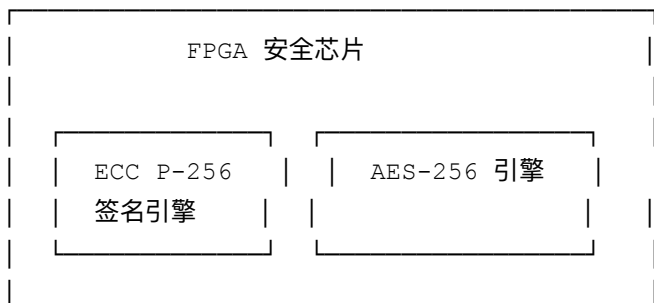
ACE 验证 attestation 的合法性 → 通过 ✓

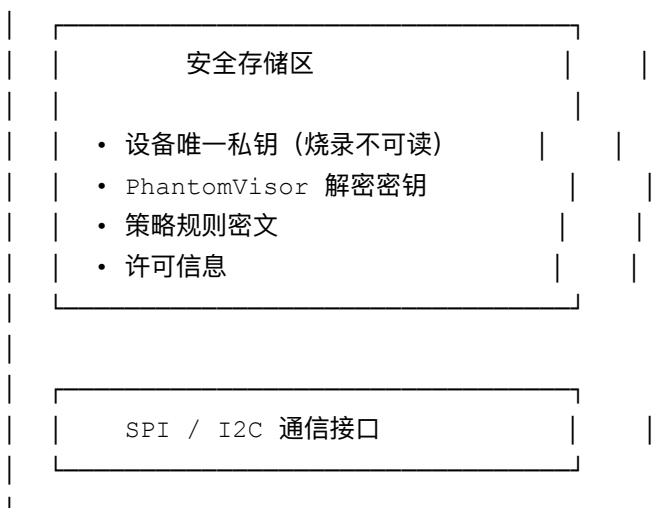
因为签名确实来自真实的 Apple Secure Enclave

七、FPGA 安全芯片——保护 PhantomVisor 自身

PhantomVisor 是整个系统的核心代码资产。安全芯片保护它不被复制和逆向。

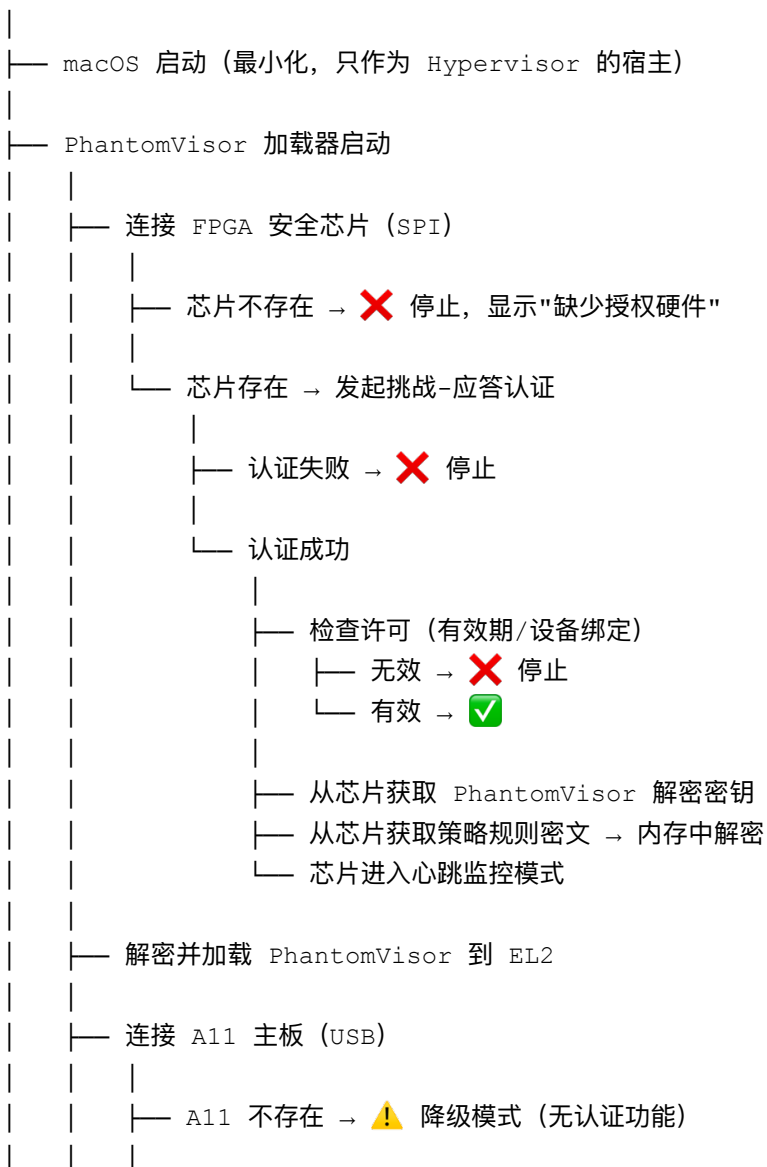
7.1 芯片内部





7.2 启动链路

M1 上电




```

|   |   └─ A11 就绪 → 建立 USB 通信通道
|   |       └─ 启动认证代理服务 ✓
|   |
|   └─ PhantomVisor 初始化完成
|
└─ 创建 VM 并启动虚拟 iOS
    |
    |   └─ 加载 iOS 内核镜像到 VM 内存
    |   └─ 配置 Stage 2 页表（隐藏 EL2 内存区域）
    |   └─ 配置虚拟设备（iPad 设备树）
    |   └─ 设置系统寄存器陷入规则
    |   └─ 启动 VM → iOS 开始引导
    |
    └─ iOS 启动完成 → 用户看到 iPad 界面
        |
        |   └─ 触摸屏输入 → PhantomVisor → VM 内 iOS
        |   └─ 画面输出 → GPU → USB-C 屏幕
        |   └─ FairPlay/Attest 请求 → 陷入 → A11
        └─ 定期心跳 → FPGA 安全芯片

```

八、PhantomVisor 利用 Apple Hypervisor.framework

Apple Silicon 的 M1 芯片支持硬件虚拟化（EL2），并且 Apple 提供了 `Hypervisor.framework` API。PhantomVisor 不需要从零写 Hypervisor，可以基于这个框架构建：

```

PhantomVisor
|
└─ 使用 Hypervisor.framework 的 API:
    |
    |   └─ hv_vm_create()           // 创建虚拟机
    |   └─ hv_vm_map()             // 映射内存到 VM (Stage 2)
    |   └─ hv_vcpu_create()        // 创建虚拟 CPU
    |   └─ hv_vcpu_run()           // 运行 VM
    |   └─ hv_vcpu_get_sys_reg()   // 读取 VM 的系统寄存器
    |   └─ hv_vcpu_set_sys_reg()   // 设置 VM 的系统寄存器（伪装）
    |   └─ hv_vcpu_get_trap_config() // 配置哪些操作触发陷入
    |   └─ hv_vcpu_exit_info()     // 获取陷入原因
    |
    └─ 自研部分:
        |
        |   └─ 陷入处理逻辑 (trap_handler.c)
        |       → 解析陷入原因，调用对应的虚拟设备模块
        |
        └─ 虚拟设备层 (virt_*.c)

```

- 构造 iPad 的设备信息返回给 VM
- └─ A11 桥接 (a11_bridge.c)
 - 把 SEP 请求通过 USB 转发给真实硬件
- └─ 策略引擎 (policy_engine.c)
 - 从 FPGA 加载规则，决定每个陷入的处理方式
- └─ vphone-cli 内核镜像
 - 复用 vphone-cli 项目的 iOS 内核和根文件系统
 - 已经有完整的越狱环境 + Sileo + TrollStore

Hypervisor.framework 的优势

优势	说明
不需要 KEXT	纯用户态 API，不涉及内核扩展
Apple 官方支持	不会被系统更新破坏
SIP 兼容	不需要关 SIP（但 AMFI 仍需绕过来加载未签名 VM）
性能接近原生	硬件虚拟化，非软件模拟
vphone-cli 已验证	vphone-cli 就是基于这个框架，说明 iOS 可以在 VM 里跑

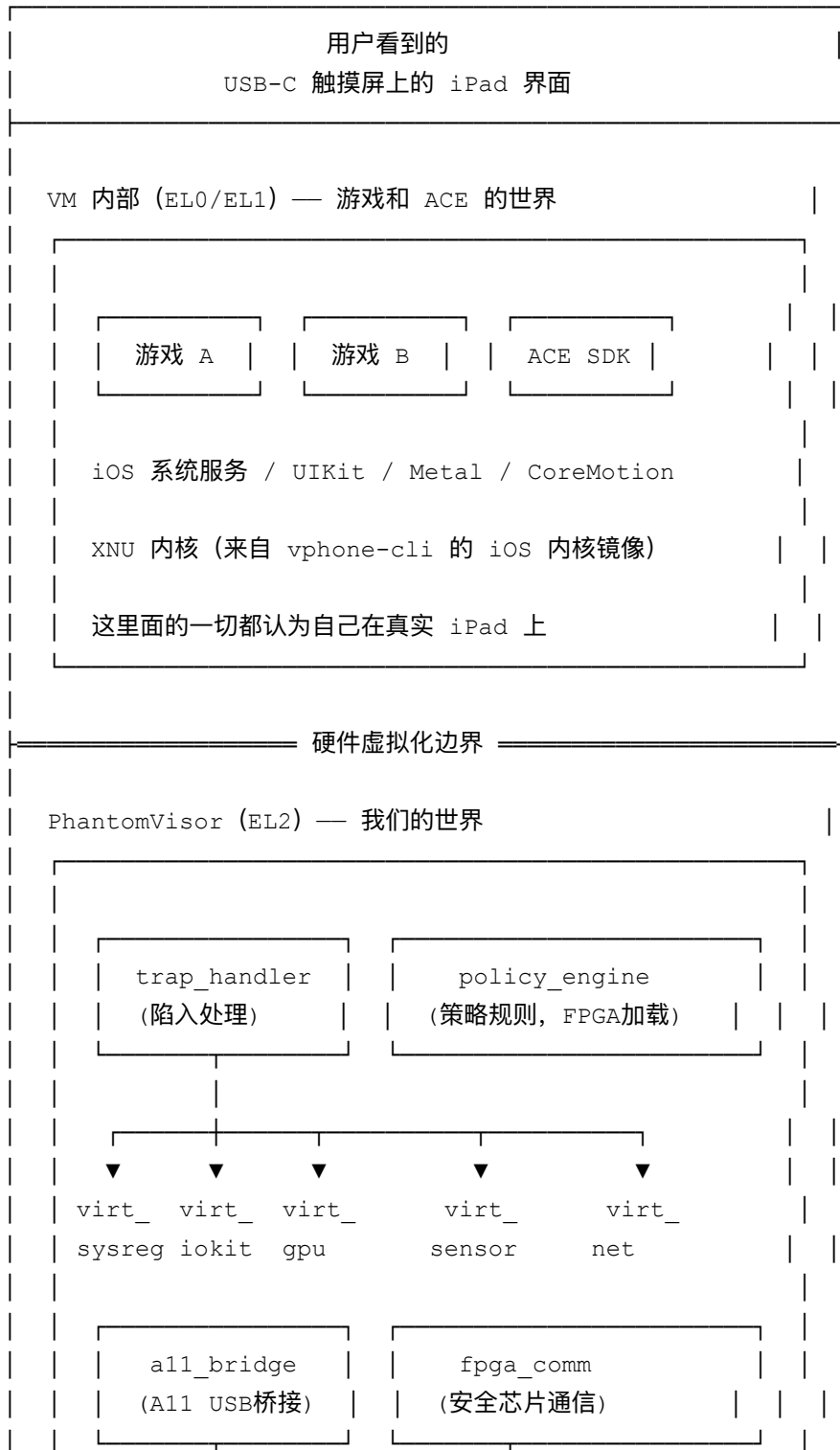
九、对比：为什么 vphone-cli 做不到但 PhantomVisor 能做到

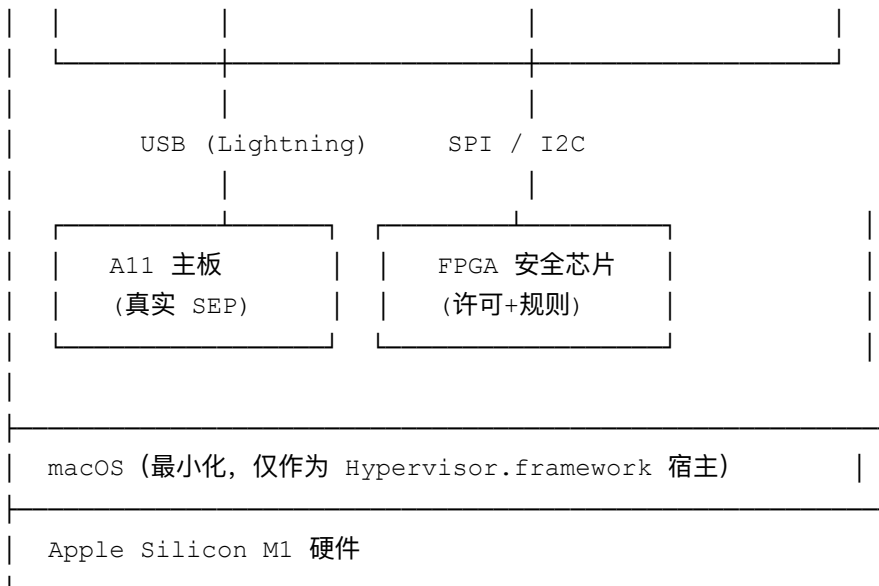
vphone-cli 和 PhantomVisor 都用 Hypervisor.framework，但关键区别：

	vphone-cli	PhantomVisor
定位	通用 iOS 虚拟机	专为不可检测设计的定制 Hypervisor
设备身份	iPhone99,11（假设备）	精确模拟真实 iPad 型号
FairPlay	依赖 VirtIO 桥接（不稳定）	A11 真实硬件桥接（硬件保证）
App Attest	不支持	A11 SEP 真实签名
Apple ID	登录不了	通过 A11 的已登录 Apple ID
反检测	没做	系统寄存器/设备树/计时器全面伪装

PhantomVisor = vphone-cli 的虚拟化能力 + PhantomOS 的反检测经验 + A11 的真实硬件认证

十、软件栈全景





十一、检测不可能性证明

ACE 能做的所有检测手段 vs PhantomVisor 的应对

ACE 检测手段	传统方案 (hook) 能防吗	PhantomVisor 能防吗	原因
读 hw.machine 等 sysctl	✅ 但可能漏	✅ 硬件级拦截	系统调用陷入 EL2
扫描 dylib image list	❌ 被发现	✅ 无额外 dylib	VM 内不存在注入
检查 __interpose section	❌ 被发现	✅ 无 interpose	VM 内不存在 hook
检查 DYLD_INSERT env	❌ 有痕迹	✅ 无注入	VM 内无环境变量
内存扫描 Mach-O headers	❌ 可能被发现	✅ 无异常 image	VM 内零注入
读 ARM 系统寄存器	❌ 无法 hook	✅ EL2 陷入伪装	硬件虚拟化
IOKit 设备树查询	✅ 但可能漏	✅ 虚拟设备树	MMIO 陷入
App Attest 签名验证	❌ 做不到	✅ A11 真实签名	真实 SEP 硬件
FairPlay DRM 校验	❌ 做不到	✅ A11 真实解密	真实 SEP 硬件

Metal GPU 参数检查	❌ 没做	✅ 虚拟 GPU 参数	陷入伪装
传感器数据检查	❌ 没做	✅ 虚拟传感器	模拟数据注入
网络栈 TLS 指纹	❌ 没做	✅ 虚拟网络参数	VM 内是 iOS 网络栈
AI 行为分析	❌ 没做	⚠️ 仍需触控模拟	唯一需要额外处理的点
进程外检测	⚠️ sysmond 防	✅ 无进程外痕迹	所有代码在 EL2
时间差检测虚拟化	不适用	✅ 计时器伪装	控制 CNTPCT_ELO

结论：除了 AI 行为分析（需要触控输入模拟），其他所有检测手段在 Hypervisor 架构下都不可能成功。

十二、开发路线

Phase 1：验证核心可行性

任务	目标	预计周期
Hypervisor.framework 原型	用最小代码创建 VM，在 VM 里跑 iOS 内核	2 周
系统寄存器陷入验证	确认 MIDR_EL1 等寄存器在 VM 读取时可拦截	1 周
vphone-cli 内核镜像复用	把 vphone-cli 的 iOS 内核加载到自研 VM 里	2 周

Phase 2：A11 桥接

任务	目标	预计周期
A11 USB 通信协议	iPhone 8 越狱后跑认证代理，Mac 通过 USB 发送/接收	3 周
FairPlay 桥接	VM 内 FairPlay 请求 → EL2 陷入 → USB → A11 SEP → 返回	4 周
App Attest 桥接	同上路径，处理 DCAppAttestService 请求	2 周

Phase 3：虚拟设备层

--	--	--

任务	目标	预计周期
虚拟 IOKit 设备树	返回完整的 iPad 设备信息	2 周
虚拟 GPU/Metal 参数	伪装 GPU Family, 纹理参数等	1 周
虚拟传感器	加速度计/陀螺仪合理模拟数据	1 周
计时器校准	抹平虚拟化延迟	1 周

Phase 4：安全芯片

任务	目标	预计周期
FPGA 固件开发	ECC 签名 + AES 加解密 + 许可管理	4 周
PhantomVisor 加密启动	核心代码加密存储，芯片解密加载	2 周
策略规则热更新	服务端 → 芯片 → PhantomVisor 规则更新链路	2 周

Phase 5：产品化

任务	目标	预计周期
触控输入映射	触摸屏输入 → VM 内 iOS 触控事件	2 周
管理界面	macOS 端管理 App（VM 控制/App 安装/设置）	2 周
外壳设计	3D 打印，整合三块板 + 屏幕 + 散热	3 周
ACE 实战测试	在 VM 里跑暗区突围，验证 ACE 检测结果	持续

十三、物料清单

组件	型号/规格	预估成本	用途
M1 主板	Mac Mini / MacBook Air 拆机	¥800-1500	Hypervisor 宿主
iPhone 8 主板	A11 拆机（可越狱版本）	¥100-300	SEP 认证代理
FPGA 开发板	Altera MAX10 (Kryptor 参考)	¥200-500	安全芯片

便携触摸屏	13-15 寸 USB-C 多点触控	¥400-800	显示+输入
Lightning-USB-C 线	数据线	¥30	A11 连接
USB-C PD 充电器	65W+	¥50-100	主供电
散热风扇	5V 小型风扇	¥20-50	散热
3D 打印外壳	定制	¥100-300	整机外壳
合计		¥1700-3550	

十四、这个架构的技术壁垒总结

壁垒层级	技术能力	可复制性
 不可复制	FPGA 内的策略规则 + 解密密钥 每台设备的唯一硬件私钥	拿不出来 物理绑定
 极难复制	EL2 Hypervisor + A11 桥接的整体方案	需要同时精通 虚拟化 + iOS 安全 + 硬件
 高成本复制	虚拟设备层（完整 iPad 环境伪装） FairPlay/App Attest SEP 桥接协议	需要大量测试验证 需要逆向 SEP 通信格式
 可复制	Hypervisor.framework API 调用 USB 通信	Apple 有文档 标准协议

最终壁垒：代码在 EL2（ACE 看不到），规则在 FPGA（反编译拿不到），签名在 A11（没有真硬件做不到）。三层锁，缺一不可。